User-Defined Function di MATLAB

Input, output, beberapa input, dan beberapa output

Andreas Syaloom Kurniawan

Faculty of Economics and Business, Universitas Gadjah Mada

Anatomi sebuah function

```
function [out1, out2, ...] = namaFungsi(in1, in2, ...)
    % badan fungsi: perhitungan
    out1 = ...;
    out2 = ...;
end
```

- `function ... end`: pembuka dan penutup definisi.
- **Output** di kiri tanda =; lebih dari satu ditulis dalam kurung siku `[ ]`.
- **Input** di dalam kurung biasa `( )`, dipisahkan koma.
- Setiap output wajib diberi nilai di dalam badan fungsi.

Di mana function disimpan?

1. File terpisah

```
% isi file: myfun1.m
function s = myfun1(x)
    ...
end
```

Nama file harus sama dengan nama fungsi. File berada di folder kerja (Current Folder) atau di *path*.

2. Local function di dalam script

```
% isi file: latihan.m
a = myfun1(3);
disp(a)

function s = myfun1(x)
    ...
end
```

Definisi fungsi diletakkan di bagian paling bawah script (R2016b+). Hanya bisa dipanggil dari script itu.

Penamaan: nama file harus sama dengan nama function

Function `myfun1` disimpan sebagai `myfun1.m`.

- `.m` = file **kode** (script atau function).
- `.mat` = file **data** (variabel hasil `save`), bukan kode.

Jika nama tidak sama, MATLAB memakai nama file dan mengabaikan nama di baris `function`:

```
% file: hitung.m
function s = myfun1(x)
    ...
end
```

```
>> hitung(3)     % jalan: nama file
>> myfun1(3)     % error
Unrecognized function or
variable 'myfun1'.
```

Editor MATLAB memberi peringatan kuning untuk ketidakcocokan ini. Tetap samakan namanya agar tidak membingungkan.

Aturan nama function (dan nama file)

1. **Diawali huruf**, lalu huruf, angka, atau _. Tidak boleh spasi atau -.
2. **Jangan memakai nama fungsi bawaan**. Membuat `sum.m` atau `mean.m` sendiri akan menimpa fungsi bawaan MATLAB.
3. **File harus terlihat oleh MATLAB**: berada di Current Folder atau di path. Jika tidak, MATLAB tidak menemukannya walaupun namanya benar.

Nama file	Status
`myfun1.m`	valid
`hitung_pdrb.m`	valid
`my-fun.m`	tidak valid (-)
`2fun.m`	tidak valid (diawali angka)
`sum.m`	berbahaya (menimpa bawaan)

Cek sebuah nama sebelum dipakai:

```
>> which sum
built-in (...\toolbox\matlab\datafun\sum)
>> which myfun1
'myfun1' not found.      % aman dipakai
```

Beberapa function dalam satu file: function utama dan local function

```
% file: myfun1.m
function s = myfun1(x)    % function utama
    y = bantu(x);
    s = log(y) - sqrt(y);
end

function y = bantu(x)     % local function
    y = x.^2 + 2*x - 4;
end
```

```
>> myfun1(3)      % jalan
ans =
   -0.9187
>> bantu(3)       % error dari luar
Unrecognized function or variable 'bantu'.
```

- Hanya function pertama yang harus bernama sama dengan file. Function ini bisa dipanggil dari luar.
- Function berikutnya adalah local function: namanya bebas, tetapi hanya bisa dipakai **di dalam file itu**.
- Hal yang sama berlaku untuk local function di bagian bawah **script**.

Gunanya: memecah perhitungan panjang menjadi bagian kecil yang rapi tanpa membuat banyak file.

Contoh 1: satu input, satu output

```
function s = myfun1(x) % output s, input x
    y = x*x + 2*x - 4;
    z = log(y);
    s = z - sqrt(y);
end
```

Secara matematis:

$$y = x^2 + 2x - 4, \quad z = \ln y, \quad s = z - \sqrt{y}$$

```
>> myfun1(3)
ans =
   -0.9187
>> a = myfun1(3);  % disimpan
```

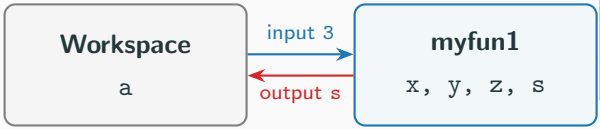
Cek manual untuk $x = 3$:

$$y = 9 + 6 - 4 = 11$$

$$z = \ln 11 = 2.3979$$

$$s = 2.3979 - 3.3166 = -0.9187$$

Variabel di dalam function bersifat lokal



Hanya nilai **output** yang kembali ke workspace.

```
>> a = myfun1(3);
>> y
Unrecognized function or
variable 'y'.
```

y dan z hanya hidup selama fungsi berjalan, lalu hilang. Nama s di dalam fungsi juga tidak harus sama dengan nama variabel penampung (a).

Contoh 2: satu input, beberapa output

```
function [s, y, z] = myfun2(x)
    y = x*x + 2*x - 4;
    z = log(y);
    s = z - sqrt(y);
end
```

Perhitungannya sama dengan `myfun1`, tetapi `y` dan `z` sekarang ikut dikembalikan sebagai output.

```
>> [s, y, z] = myfun2(3)
s =
   -0.9187
y =
    11
z =
    2.3979
```

Mengambil sebagian output

```
>> s = myfun2(3)        % output ke-1
s =
   -0.9187
>> [s, y] = myfun2(3)   % ke-1 dan ke-2
>> [~, y] = myfun2(3)   % lewati s
y =
    11
>> [~, ~, z] = myfun2(3)
z =
    2.3979
```

- Output diambil berurutan dari kiri, sesuai urutan di `[s, y, z]`.
- Tanda `~` membuang output yang tidak dibutuhkan.
- Yang menentukan urutan adalah **posisi**, bukan nama: `[a, b] = myfun2(3)` memberi `a = s` dan `b = y`.

Contoh 3: beberapa input, satu output

```
function mu = myfun3(s, t)   % output mu, input s dan t
    nilai = s.^2 + 2*t - log(10);
    mu = exp(nilai);
end
```

Secara matematis:

$$\mu = \exp\left(s^2 + 2t - \ln 10\right) = \frac{e^{s^2+2t}}{10}$$

```
>> myfun3(1, 2)
ans =
   14.8413
>> myfun3(2, 1)  % urutan!
ans =
  40.3429
```

Input dicocokkan berdasarkan posisi: argumen pertama masuk ke `s`, argumen kedua ke `t`.

Operator titik: function yang menerima vektor

myfun3 memakai `s.^2` (per elemen), sehingga bisa menerima vektor:

```
>> myfun3([1 2], 2)
ans =
   14.8413  298.0958
```

myfun1 memakai `x*x` (perkalian matriks):

```
>> myfun1([1 2 3])
Error using  *
Incorrect dimensions for matrix
multiplication. ...
```

Perbaikan: ganti dengan operator per elemen.

```
function s = myfun1v(x)
    y = x.^2 + 2*x - 4;
    z = log(y);
    s = z - sqrt(y);
end
```

`log`, `sqrt`, dan `exp` sudah bekerja per elemen; yang perlu diperhatikan hanya `*`, `/`, dan `^`.

Hati-hati domain: `log` dan `sqrt`

`myfun1` hanya bernilai real jika $y > 0$:

$$x^2 + 2x - 4 > 0$$

$$\iff x < -1 - \sqrt{5} \text{ atau } x > -1 + \sqrt{5}$$

yaitu $x < -3.236$ atau $x > 1.236$.

Di luar itu MATLAB tidak error, tetapi diam-diam mengembalikan bilangan **kompleks**.

```
>> myfun1(0)        % y = -4
ans =
   1.3863 + 1.1416i
>> isreal(myfun1(0))
ans = 0
```

Bisa ditambahkan pengecekan di awal fungsi:

```
if any(y <= 0)
    error('y harus > 0');
end
```

Alternatif singkat: anonymous function

Untuk rumus satu baris, tidak perlu file terpisah:

```
>> f = @(s,t) exp(s.^2 + 2*t - log(10));
>> f(1, 2)
ans =
   14.8413
```

`f` di sini setara dengan `myfun3`.

`function`	anonymous `@`
banyak baris	satu ekspresi
banyak output	satu output
file / bagian bawah script	di mana saja

Ringkasan

Pola	Definisi	Pemanggilan
1 input, 1 output	`function s = myfun1(x)`	`a = myfun1(3)`
1 input, banyak output	`function [s,y,z] = myfun2(x)`	`[s,y,z] = myfun2(3)`
banyak input, 1 output	`function mu = myfun3(s,t)`	`mu = myfun3(1,2)`
lewati output	—	`[~,y] = myfun2(3)`

- Input dan output dicocokkan berdasarkan posisi.
- Variabel di dalam fungsi bersifat lokal.
- Gunakan `.*`, `./`, `.^` agar fungsi menerima vektor.

Handling Complex Input

Input tidak selamanya berupa satu angka

Sejauh ini `myfun1`, `myfun2`, dan `myfun3` menerima **satu angka**, misalnya `myfun1(3)`. Di pekerjaan nyata, input sebuah fungsi sering datang dalam bentuk lain:

Bentuk input	Contoh di ekonomi	Wadah MATLAB
Vektor	inflasi bulanan 2015–2024 (120 observasi)	array 120×1
Matriks	PDRB 38 provinsi $\times$ 10 tahun	array 38×10
List	harga beberapa komoditas, panjang data berbeda	cell array
Record	parameter model: β, σ, α	struct
Data tabular	hasil survei dari CSV/Excel	table
Teks	nama provinsi, nama file	string / char
Fungsi	rumus yang akan diintegralkan atau dioptimasi	function handle `@`

Section ini fokus pada dua bentuk yang paling sering muncul: list (cell array) dan matriks.

Tiga pertanyaan sebelum menulis function

1. **Input-nya berbentuk apa?**
   Angka, vektor, matriks, cell, atau struct? Berapa ukurannya?
2. **Output-nya ingin berbentuk apa?**
   Satu angka (ringkasan), array (satu nilai per item), atau matriks (beberapa nilai per item)?
3. **Bagaimana cara "berjalan" di dalam input?**
   Loop per elemen/sel, argumen `dim`, atau operasi vektor?

Jawaban pertanyaan 2 menentukan cara *preallocate*; jawaban pertanyaan 3 menentukan cara mengisi output.

Cara memeriksa bentuk input:

```
>> class(x)      % tipe: 'double', 'cell'
>> size(x)       % ukuran
>> numel(x)      % jumlah elemen/sel
>> iscell(x)     % cell?
>> isnumeric(x)  % angka?
```

Satu fungsi bisa memakai pengecekan ini untuk menolak input yang salah bentuk:

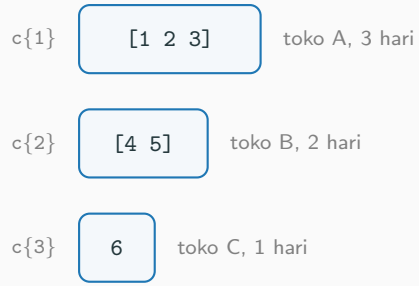
```
if ~iscell(c)
    error('Input harus cell array');
end
```

Data contoh (1): list `c`

Kita buat list (cell array) berisi tiga sel:

```
>> c = {[1 2 3], [4 5], 6}
c =
  1x3 cell array
    {[1 2 3]}    {[4 5]}    {[6]}
>> c{1}      % isi sel 1: [1 2 3]
>> c{2}      % isi sel 2: [4 5]
>> c{3}      % isi sel 3: 6
```

Kurung kurawal `c{i}` membuka isi sel ke-i.



- 3 sel dengan panjang berbeda: 1×3, 1×2, 1×1.
- `numel(c)` $= 3$ sel; total angkanya 6.
- Karena panjangnya berbeda, data ini tidak bisa disimpan sebagai satu matriks.

Data contoh (2): matriks `A`

```
>> A = magic(3)
A =
     8     1     6
     3     5     7
     4     9     2
```

	thn 1	thn 2	thn 3
prov 1	8	1	6
prov 2	3	5	7
prov 3	4	9	2

Cerita: baris = provinsi, kolom = tahun.

Misalnya `A(2,3) = 7` adalah nilai provinsi 2 pada tahun 3.

Input \ Output	angka	array	matriks
list `c`	`totalList`	`jumlahPerItem`	`statPerItem`
matriks `A`	`rentang`	`maksPerBaris`	`standarkan`

Peta enam contoh berikut: setiap kombinasi bentuk input dan bentuk output.

List → angka: `totalList`

Data: `c = {[1 2 3], [4 5], 6}` (sel 1 = toko A, sel 2 = toko B, sel 3 = toko C)
Masalah: penjualan harian tiga toko dengan jumlah hari buka berbeda.
Dicari: total penjualan semua toko digabung → satu angka.

```
function t = totalList(c)
    t = 0;
    for i = 1:numel(c)
        t = t + sum(c{i}, 'all');
    end
end
```

```
>> totalList(c)
ans =
    21
```

- `numel(c)` = jumlah sel = 3.
- `c{i}` membuka isi sel ke-i.
- Penjumlahan bertahap: $0 + 6 + 9 + 6 = 21$.

Jebakan: `sum(c(i))` error, karena `c(i)` masih berupa cell.

List → array: jumlahPerItem

Data: `c = {[1 2 3], [4 5], 6}` (sel 1 = toko A, sel 2 = toko B, sel 3 = toko C)
Masalah: data yang sama, tetapi sekarang kita ingin membandingkan antar toko.
Dicari: total penjualan per toko → satu angka per toko (array).

```
function v = jumlahPerItem(c)
    v = zeros(1, numel(c));   % preallocate
    for i = 1:numel(c)
        v(i) = sum(c{i}, 'all');
    end
end
```

```
>> jumlahPerItem(c)
ans =
     6     9     6
```

- Satu angka per sel → vektor sepanjang `numel(c)`.
- `zeros(1,n)` membuat hasil berupa vektor baris.

Versi singkat tanpa loop:

```
>> cellfun(@(x) sum(x,'all'), c)
ans =
     6     9     6
```

List → matriks: `statPerItem`

Data: `c = {[1 2 3], [4 5], 6}` (sel 1 = toko A, sel 2 = toko B, sel 3 = toko C)
Masalah: total saja tidak cukup; kita ingin profil singkat tiap toko.
Dicari: untuk tiap toko: jumlah hari buka, rata-rata, dan penjualan tertinggi → matriks (baris = toko, kolom = statistik).

```
function M = statPerItem(c)
    M = zeros(numel(c), 3);
    for i = 1:numel(c)
        x = c{i}(:);
        M(i, :) = [numel(x), mean(x), max(x)];
    end
end
```

- `c{i}(:)` mengubah isi sel menjadi vektor kolom, aman untuk sel berbentuk matriks.
- `M(i,:)` mengisi baris ke-i sekaligus.

```
>> statPerItem(c)
ans =
    3.0000    2.0000    3.0000
    2.0000    4.5000    5.0000
    1.0000    6.0000    6.0000
```

Baca per baris: toko A buka 3 hari, rata-rata 2, tertinggi 3; toko B 2 hari, rata-rata 4.5, tertinggi 5; toko C 1 hari dengan penjualan 6.

List matriks → list: `barisUjung`

Data: `L = {magic(4), [1 2; 3 4; 5 6], [7 8 9]}` (ukuran 4×4, 3×2, 1×3)

Masalah: beberapa matriks berukuran *berbeda-beda*, tetapi berhingga.

Dicari: baris pertama dan terakhir dari setiap matriks, berapa pun ukurannya.

```
function out = barisUjung(L)
    out = cell(size(L));
    for i = 1:numel(L)
        M = L{i};
        if isempty(M)
            out{i} = M;
        else
            out{i} = M([1 end], :);
        end
    end
end
```

```
>> out = barisUjung(L)
out =
  1x3 cell array
    {2x4 double} {2x2 double} {2x3 double}
>> out{1}
ans =
    16     2     3    13
     4    14    15     1
```

`M([1 end], :)` = baris ke-1 dan ke-end, semua kolom; `end` menyesuaikan jumlah baris tiap matriks.

barisUjung: catatan dan versi singkat

- **Kenapa output berupa cell?** Jumlah kolom hasil berbeda (4, 2, 3), sehingga tidak bisa ditumpuk menjadi satu matriks.
- **Matriks satu baris**: `out{3}` berisi `[7 8 9; 7 8 9]`, baris yang sama dua kali. Jika tidak diinginkan: `M(unique([1 size(M,1)]), :)`.
- **Matriks kosong**: `M([1 end],:)` error, karena itu dicek dengan `isempty` lebih dulu.
- Jika semua matriks punya jumlah kolom sama, hasilnya bisa ditumpuk: `vertcat(out{:})`.

Versi satu baris dengan `cellfun`

```
out = cellfun(@(M) M([1 end], :), L, ...
              'UniformOutput', false);
```

`'UniformOutput', false` wajib karena tiap hasil berupa matriks, bukan satu angka. Tanpa opsi ini `cellfun` error.

Versi ini tidak menangani matriks kosong; versi dengan loop lebih aman untuk input yang belum pasti bentuknya.

Matriks → angka: `rentang`

Data: `A = [8 1 6; 3 5 7; 4 9 2]` (baris = provinsi, kolom = tahun)
Masalah: suatu indikator untuk 3 provinsi selama 3 tahun.
Dicari: seberapa lebar sebaran seluruh nilai, yaitu nilai tertinggi dikurangi terendah → satu angka.

```
function r = rentang(A)
    r = max(A, [], 'all') - min(A, [], 'all');
end
```

```
>> rentang(A)
ans =
     8
```

- `'all'` meringkas seluruh elemen menjadi satu angka: $9 - 1 = 8$.
- Tanpa `'all'`, `max(A)` memberi maksimum per kolom (vektor 1×3).
- Argumen kedua `[]` wajib diisi agar `'all'` terbaca sebagai opsi dimensi.

Matriks → array: `maksPerBaris`

Data: `A = [8 1 6; 3 5 7; 4 9 2]` (baris = provinsi, kolom = tahun)
Masalah: kita ingin melihat capaian tiap provinsi.
Dicari: nilai tertinggi yang pernah dicapai tiap provinsi → satu angka per baris (array).

```
function v = maksPerBaris(A)
    v = max(A, [], 2);     % per baris
end
```

```
>> maksPerBaris(A)
ans =
     8
     7
     9
```

- Argumen `dim` bekerja seperti pada `sum`: dim 2 meringkas sepanjang kolom, hasilnya satu angka per baris.
- Hasilnya vektor kolom 3×1.

Orientasi: `jumlahPerItem` menghasilkan $1 \times n$, sedangkan `maksPerBaris` menghasilkan $m \times 1$. Samakan dulu (misalnya dengan `'`) sebelum digabung.

Matriks → matriks: `standarkan`

Data: `A = [8 1 6; 3 5 7; 4 9 2]` (baris = provinsi, kolom = tahun)

Masalah: nilai mentah sulit dibandingkan antar tahun.

Dicari: posisi relatif tiap provinsi dalam tahunnya (z-score per kolom) → matriks berukuran sama.

```
function Z = standarkan(A)
    Z = (A - mean(A)) ./ std(A);
end
```

```
>> standarkan(A)
ans =
    1.1339   -1.0000    0.3780
   -0.7559         0    0.7559
   -0.3780    1.0000   -1.1339
```

Z-score per kolom:

$$z_{ij} = \frac{a_{ij} - \bar{a}_j}{s_j}$$

- `mean(A)` dan `std(A)` berukuran 1×3.
- Implicit expansion (R2016b+) memperluasnya ke setiap baris A (3×3).
- Ukuran output sama dengan input.

`numel`: jumlah elemen

`numel` = *number of elements*: jumlah total elemen array, apa pun bentuknya.

```
>> numel([1 2 3])          % 3
>> numel(magic(4))         % 16 = 4*4
>> numel(zeros(3,4,5))     % 60 = 3*4*5
>> numel(c)                % 3 (sel)
```

Pada cell array, yang dihitung adalah jumlah sel, bukan jumlah angka di dalamnya (6).

Fungsi	`magic(4)`	Arti
`numel`	16	total elemen
`size`	`[4 4]`	ukuran tiap dimensi
`length`	4	dimensi terbesar

Untuk loop atas semua elemen, gunakan `numel`. `length(zeros(3,100))` bernilai 100, bukan 300.

Ringkasan: handling complex input

- Jenis output ditentukan oleh nilai terakhir variabel output, bukan oleh deklarasi; MATLAB tidak punya *return type*.
- Input list (cell): loop dengan `for i = 1:numel(c)` dan buka isinya dengan `c{i}`.
- Input matriks: manfaatkan argumen `dim` atau `'all'` pada `sum`, `max`, `min`, `mean`.
- Preallocate output: `zeros(1,n)` (baris), `zeros(n,1)` (kolom), `zeros(n,k)` (matriks).
- Perhatikan orientasi vektor hasil sebelum digabung.
- `cellfun` dan operasi vektor sering bisa menggantikan loop.